ROLE-SPECIFIC PREP · RECRUITER SCREEN + 30-MINUTE TECHNICAL

GenAI Engineer

Interview Cheat Sheet

Screening Calls & Last-Minute Interview Prep

Prompting, structured outputs, multimodal generation, safety filters, streaming UX, and cost control for generative product features.

10 PAGES

15 MUST-KNOW QUESTIONS

30 MINUTE MOCK INTERVIEW

1 ROLE-SPECIFIC SYSTEM DESIGN



INTERVIEW PREPARATION · 10 PAGES · RECRUITER SCREEN + 30-MINUTE TECHNICAL

GenAI Engineer

Screening Calls & Last-Minute Interview Prep

PAGE 1 OF 10

1 Role Overview

responsibilities · daily work · competencies · metrics

A GenAI Engineer builds product features that **generate** content: text, images, summaries, code, or structured drafts. Most work uses foundation models through APIs. The hard parts are control and quality: getting on-brand, safe, correct output at a predictable cost and latency, and proving that a prompt or model change made things better. Training models from scratch is not part of the job.

What you own and what a normal week looks like

Area	Concrete daily work
Prompt and output design	Write system prompts, few-shot examples, and output schemas. Version every prompt like code.
Generation pipelines	Chain steps: plan, draft, check, revise. Combine text and image models. Add grounding data where facts matter.
Quality evaluation	Build rubric-based evals, pairwise comparisons, and LLM judges calibrated against human ratings.
Safety and policy	Input and output moderation, banned-claim checks, PII handling, copyright and brand rules.
User experience	Streaming, regenerate, edit-in-place, partial results, clear error states, feedback buttons.
Cost and latency	Token budgets, model routing, caching, batch generation for bulk jobs.
Experimentation	A/B test prompts and models on user metrics: acceptance rate, edit distance, retention.

Required competencies

Must have	Differentiators
• Python or TypeScript backend skills	• Image models: diffusion, editing, inpainting
• Prompting, few-shot, structured output	• Vision-language input (screenshots, documents)
• LLM API mechanics: tokens, sampling, limits	• Judge calibration and pairwise preference evals
• Evaluation of open-ended output	• Prompt caching and cost modelling
• Streaming APIs (SSE, WebSockets)	• Brand-voice control and style transfer
• Content safety and moderation basics	• IP, licensing, and disclosure requirements

Success metrics (report each against a baseline)

Metric	What it tells you
Acceptance rate (used without major edit)	The single best product signal for generated content.
Edit distance after acceptance	How much users fix. High edits mean low trust.
Rubric score and judge win rate	Offline quality on a fixed set before release.
Policy violation rate	Safety: unsafe, off-brand, or banned-claim outputs per 10k.
Time to first token, cost per generation	UX and unit economics.

KEY INSIGHT **What interviewers evaluate:** (1) can you turn "make it good" into a measurable rubric, (2) do you control output with structure and checks rather than longer prompts, (3) do you treat safety and cost as design inputs, (4) do you connect offline scores to user metrics. **Strong** answers include an eval method and a number. **Weak** answers describe prompt tweaks.

PAGE 2 OF 10

Core Technical Skills

the concepts every answer builds on

Concept	Say this	Common gap
Tokens and limits	Cost and latency scale with input and output tokens. Output tokens are slower and dearer. Context window limits total size. Count tokens before you call.	Ignoring output length.
Sampling	Temperature and top-p control variety. Use low values for extraction and classification, higher for ideation. Seeds do not guarantee identical output across versions.	One temperature for all tasks.
Prompt structure	Role and goal, constraints, context, examples, output format. Put stable parts first to benefit from prompt caching. Keep instructions testable.	Long prose prompts with no structure.
Structured output	JSON schema or tool-calling mode, validated with Pydantic or Zod. Generate structure first, then fill prose fields.	Regex on free text.
Decomposition	Split hard tasks: outline, draft, critique, revise. Each step is easier to evaluate. Stop when extra steps no longer raise the score.	One giant prompt for everything.
Grounding	Facts come from supplied data (product catalog, docs), not model memory. Ask for citations or field references and verify them.	Trusting the model for facts.
Evaluating open text	Rubric with 3 to 6 criteria, each scored 1 to 5 with anchors. Pairwise comparison for close variants. LLM judge only after agreement with humans is measured.	"It reads better."
Image generation	Diffusion models; control with prompt, negative prompt, reference image, seed, steps, guidance scale. Inpainting edits regions. Evaluate with human review and automated checks (text, logos, faces).	No plan for artefacts or IP.
Multimodal input	Images, PDFs, and audio enter as tokens too. Resize images to the detail needed; extract text first when OCR is cheaper and more exact.	Sending 20 full-size images per call.
Prompt caching	Providers reuse a stable prompt prefix at lower cost and latency. Order content: system rules, style guide, examples, then the variable input.	Variable data at the top of the prompt.
Safety layers	Input moderation, system rules, output moderation, deterministic checks (banned terms, claims), human review for high-risk outputs.	One moderation call as the only control.
Streaming UX	Stream tokens to cut perceived latency. Buffer by sentence when output checks must run before display.	Streaming text that must be retracted.
Cost control	Route by task difficulty, cache prompts and results, cap output, use batch APIs for bulk jobs.	No cost per generation figure.

KEY INSIGHT **Control beats persuasion.** Each control you move from the prompt into code (schema, enum, length cap, banned-term check, citation check) removes a class of failure. Prompts guide the model; code enforces the rule.

GENERATION PIPELINE Input checks → retrieve facts and brand rules → generate (structured) → validate (schema, policy, facts) → repair or regenerate (bounded) → human review when risk is high → log for evals.

INTERVIEW TRAP Fixing every quality issue by adding sentences to the prompt. After a few rounds the prompt contradicts itself. Interviewers want a test set, a measured change, and checks in code.

PAGE 3 OF 10

3 Technical Interview Questions

high-probability questions with concise answers

Q01 How do you design and iterate a prompt for a new feature? PROMPTING

Write the task spec first: inputs, output schema, constraints, examples of good and bad output. Build a test set of 50 to 100 real inputs. Start with a simple prompt, run the set, score with a rubric, read failures, change one thing, and re-run. Version each prompt and keep the scores.

Q02 How do you evaluate generated text when there is no single right answer? QUALITY

Define a rubric with anchored levels: accuracy against source, coverage, tone, format, policy. Use pairwise comparison between versions because humans judge relative quality better. Calibrate an LLM judge against 100+ human ratings and report agreement. Track online acceptance and edit rate.

Q03 What are the risks of LLM-as-judge? JUDGES

Position bias, length bias, self-preference, and drift when the judge model changes. Mitigate: swap order and average, score criteria separately, use reference answers when they exist, pin the judge version, and re-check agreement with humans on a fresh sample each quarter.

Q04 How do you reduce hallucinations in generated content? FACTS

Supply the facts (catalog fields, retrieved docs) and tell the model to use only those. Ask for field references or citations. Verify claims in code: numbers, names, and prices must match the source. Allow "unknown". Measure an unsupported-claim rate on the test set.

Q05 The model breaks the JSON format 3% of the time. Fix it. STRUCTURE

Use native structured-output or tool-calling mode. Simplify the schema: enums, fewer nested optional fields. Validate and send the error back for one repair attempt. Cap input length. Track the failure rate per prompt version and model.

Q06 How do you keep generated content on-brand and compliant? SAFETY

Put brand rules in a versioned style guide used by the prompt and the checker. Enforce banned terms and claim rules in code. Run output moderation. Route regulated categories (health, finance) to human review. Sample outputs weekly and add violations to the test set.

Q07 How do you build a reliable image-generation feature? IMAGES

Constrain inputs: templates, reference images, fixed aspect ratios. Fix seeds for reproducible review. Check outputs automatically for text artefacts, faces, and logos; run safety classifiers. Keep provenance metadata. Review licensing of the model and training data for commercial use.

Q08 Users say generation feels slow. What do you do? LATENCY

Measure time to first token and total time. Stream output. Shorten the prompt and cache its stable prefix. Cap output length. Generate a short draft first and expand on request. Use a smaller model for simple sections. Run independent sections in parallel.

Q09 How do you cut generation cost without losing quality? COST

Find cost per feature and per step. Route easy requests to a smaller model, cache repeated generations, enable prompt caching, cap output tokens, and use batch APIs for bulk jobs. Confirm with the rubric and pairwise win rate that quality holds.

Q10 How do you ship a new prompt or model version? RELEASE

Run the offline set and compare per segment with pairwise judging. Check safety metrics. Shadow or canary to a small user share and watch acceptance, edit rate, and violations. Keep a flag to roll back. Record the version on every output.

Q11 When do you fine-tune instead of prompting? CHOICE

When you need a consistent style or format that prompts cannot hold, when a smaller model must match a larger one for cost, or when the prompt is very long. You need 1k+ high-quality examples and an eval set. Facts belong in retrieval, not fine-tuning.

Q12 Users try to make the feature produce harmful content. How do you respond? ABUSE

Moderate input and output, rate-limit per user, log and review flagged sessions, and keep system rules short and firm. Treat jailbreak prompts found in logs as test cases. Do not rely on the prompt alone; enforce limits in code and policy.

KEY INSIGHT **Strong vs weak:** strong answers name the eval method and the control in code. Weak answers say "I would tweak the prompt".

PAGE 4 OF 10

Tools and Technologies

what to use, when, and what to watch for

Category	Options	Use when	Watch out
Text models	OpenAI, Anthropic, Google Gemini, Mistral; Bedrock, Vertex, Azure	Drafting, rewriting, summarising, structured generation.	Version changes alter tone; pin versions.
Image models	DALL-E, Imagen, Stable Diffusion (SDXL), Flux; ControlNet	Marketing images, variants, inpainting, product scenes.	Licensing, text artefacts, likeness rights.
Vision input	GPT-4o-class, Claude, Gemini vision	Screenshots, documents, charts as input.	OCR-type errors on small text.
Speech	Whisper, cloud TTS and STT	Voice input, narration, transcripts.	Latency, accents, consent for voices.
Structured output	Pydantic, Zod, Instructor, provider schema modes	Any output read by code.	Valid JSON with wrong values.
Prompt management	Git, Langfuse, PromptLayer, internal registry	Versioning, diffing, rollback of prompts.	Prompts edited in production with no review.
Evaluation	promptfoo, DeepEval, OpenAI Evals, custom pairwise harness	Rubric scoring, judge runs, CI gates.	Uncalibrated judges.
Safety	Provider moderation APIs, Llama Guard, Presidio, custom classifiers	Input and output filtering, PII.	False positives that block normal users.
Serving	FastAPI, Node, Next.js, SSE, Vercel AI SDK	Streaming UIs and APIs.	Timeouts on long streams; retries mid-stream.
Observability	Langfuse, LangSmith, Helicone, OpenTelemetry	Traces, cost, latency, prompt versions.	Logging user content without consent.
Orchestration	Plain Python, LangGraph, LlamaIndex workflows	Multi-step plan, draft, check, revise pipelines.	Framework hides prompts you must debug.
Experiments	LaunchDarkly, Statsig, GrowthBook	A/B tests of prompts and models on acceptance.	Peeking; no guardrail metrics.
Batch	Provider batch APIs, queues (SQS, Celery)	Bulk catalog generation overnight.	Partial failures; idempotent writes.

Choosing a model: four questions to answer out loud

1. **What does the rubric say on our test set?** Public benchmarks do not measure your brand voice.
2. **What are the latency and cost per generation?** Multiply by daily volume.
3. **What are the data, licensing, and residency terms?** Check commercial use for images.
4. **Can we switch?** Keep prompts, schemas, and evals provider-neutral behind one gateway.

BUILD VS BUY FOR IMAGES Hosted image APIs: fast, licensed for commercial use under their terms, little control. Self-hosted open models (SDXL, Flux): fine-tune on brand assets with LoRA, full control, but GPU cost and your own licence review. Decide on volume, brand-control needs, and legal terms.

KEY INSIGHT A safe interview line: "The test set and rubric are the asset. Models and prompts change every quarter; the evaluation lets us change them safely."

INTERVIEW TRAP Naming the newest model as the answer. Say how you would compare it with the current one on your own set.

PAGE 5 OF 10

Coding and Practical Skills

exercises, debugging, and what interviewers watch

Exercise 1: streaming endpoint with sentence-level output checks

PYTHON

```python
@app.post("/generate")
async def generate(req: GenRequest):
    if moderation_flagged(req.text):
        raise HTTPException(400, "input not allowed")
    async def events():
        buf, sent = "", 0
        async for tok in llm.stream(build_prompt(req), max_tokens=req.max_tokens):
            buf += tok
            while (cut := sentence_end(buf)) is not None:   # check whole sentences
                sentence, buf = buf[:cut], buf[cut:]
                if violates_policy(sentence):               # banned claims, PII
                    yield sse({"type": "stopped", "reason": "policy"}); return
                sent += len(sentence)
                yield sse({"type": "text", "data": sentence})
        if buf and not violates_policy(buf):
            yield sse({"type": "text", "data": buf})
        yield sse({"type": "done", "prompt_version": PROMPT_V})
    return StreamingResponse(events(), media_type="text/event-stream")
```

Exercise 2: pairwise judge with position swap

PYTHON

```python
def pairwise_win_rate(cases, gen_a, gen_b, judge):
    wins = ties = 0
    for c in cases:
        a, b = gen_a(c.input), gen_b(c.input)
        r1 = judge(c.input, first=a, second=b, rubric=RUBRIC)   # "first"/"second"/"tie"
        r2 = judge(c.input, first=b, second=a, rubric=RUBRIC)   # swapped order
        if r1 == "first" and r2 == "second": wins += 1
        elif r1 == r2 == "tie" or r1 != r2: ties += 1           # inconsistent = tie
    return wins / len(cases), ties / len(cases)
```

Debugging scenarios

Symptom	First check	Likely fix
Tone changed overnight	Model alias moved to a new version.	Pin version; run test set before upgrades.
Made-up product specs	Facts missing from context; no verification.	Supply catalog fields; verify numbers in code.
Outputs too long	No length cap; examples are long.	`max_tokens`, length in schema, shorter examples.
Judge prefers v2, users do not	Judge length bias; rubric misses user needs.	Re-calibrate judge; add acceptance metric gate.
Image text is garbled	Model weakness with typography.	Render text as an overlay layer, not in the image.
Same input, different quality	High temperature; no examples for edge cases.	Lower temperature; add examples; sample n and pick by judge.

KEY INSIGHT **Live-coding signals:** handle the failure path first, keep checks in code, record the prompt version, and explain how you test with a fake stream.

PAGE 6 OF 10

Project Deep Dive

how to present a generative feature in 4 to 6 minutes

Seven-beat structure

Beat	What to say
1. Problem and users	Who writes what today, how long it takes, and what "good" means to them.
2. Constraints	Brand rules, regulated claims, languages, latency, cost ceiling, review capacity.
3. Quality definition	Your rubric and test set. How you got humans to agree on it.
4. Pipeline and controls	Steps, model choice, schema, checks in code, review routing.
5. What broke	A real failure (off-brand, false claim, cost spike), detection, fix.
6. Results	Acceptance rate, edit distance, time saved, violation rate, cost.
7. Hindsight	What you would change and measure sooner.

Example: 90-second version (replace with your own numbers)

PRODUCT DESCRIPTION GENERATOR "Merchandisers wrote descriptions for 40k new products per quarter. I built a generator that takes catalog attributes and a brand style guide and returns a title, three bullets, and a paragraph in JSON. A 6-criterion rubric, labelled by two copywriters on 150 products (kappa 0.71), defined quality. Code checks block unsupported specs and banned claims. Version 1 had a 48% acceptance rate; after a plan-then-write pipeline and attribute verification it reached 79%. Unsupported-claim rate fell from 6% to 0.4%, and cost is $0.002 per product with batch generation." *(Illustrative figures; use yours.)*

Decisions worth rehearsing

Decision	Alternative	Why	Cost
JSON sections, not free text	Single paragraph	Each part testable and editable	Stiffer prose
Spec verification in code	Prompt instruction only	False specs are legal risk	Some false blocks
Plan-then-write two steps	One prompt	+14 points acceptance	1.6x latency
Batch API for catalog	Real-time calls	Half price, no rate limits	Hours of delay
Pairwise judge for release	Absolute scores	Better human agreement	Two generations per case

Numbers to have ready before the call

Acceptance	Edit rate	Violations	Latency	Cost	Volume
Before and after	Chars changed	Per 10k outputs	TTFT, total	Per generation	Per day or job

Follow-ups to expect

- How did you make two humans agree on what "good" means?
- How did you validate the LLM judge? What was its agreement with humans?
- What happened when the model version changed?
- How do you stop false claims you did not anticipate?

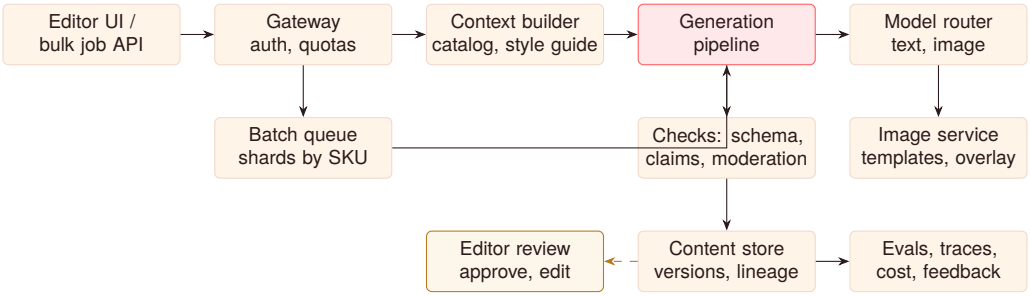
KEY INSIGHT **Strong:** a rubric, a measured lift, a code-level control, and a failure story. **Weak:** screenshots of nice outputs with no metric. Without a production project, present a side project with its test set and judge-agreement numbers.

PAGE 7 OF 10

System Design

a realistic generative feature architecture problem

Prompt: *Design a content studio for an e-commerce company. It generates product copy and lifestyle images for 2M SKUs in bulk, and on demand in an editor. Five languages. Brand and legal rules apply. Editors approve before publishing. Target: editor first token under 1 s, bulk run of 2M SKUs in 48 hours, cost under $0.01 per SKU for text.*



Solution framework (use this order out loud)

Step	What a strong answer covers
1. Clarify (2 min)	Output types and length, languages, regulated categories, review capacity, how "approved" is recorded.
2. Quality definition	Rubric per output type, test set per category and language, pairwise release gate.
3. Context and grounding	Catalog attributes and style guide injected per request; the model may not add specs absent from attributes.
4. Pipeline	Structured generation per section; verification step; bounded regenerate on failure; translation or native generation per language (test both).
5. Safety and legal	Banned terms and claims in code; moderation; regulated categories to mandatory review; image provenance metadata.
6. Two paths	Editor path: stream, small prompt, cached prefix. Bulk path: batch API, sharded queue, idempotent writes keyed by SKU and version.
7. Capacity and cost	2M SKUs in 48 h ≈ 12 per second sustained. At 1.5k tokens each, about 3B tokens: route to a cost-efficient model and use batch pricing.
8. Images	Template scenes plus product cut-out; text rendered as overlay; automated checks; human review sample.
9. Feedback	Store edits and rejections; weekly eval refresh; monitor acceptance and violations per category.

KEY INSIGHT **Distinguishing moves:** separate editor and bulk paths, facts only from catalog attributes, claim checks in code, idempotent bulk writes, and a token-volume estimate that drives model choice.

Follow-ups the interviewer will push

Push	Good response
"Style guide changes next week?"	Style guide is versioned data, not prompt text; re-run test set; regenerate only affected SKUs.
"Translate or generate natively?"	Test both per language on the rubric; native often wins on tone, translation on consistency.
"Batch job fails at 60%?"	Idempotent writes keyed by SKU and version; resume from the queue; no duplicates.
"Editors reject 40% in one category?"	Segment metrics; read rejections; category-specific examples or rules.

INTERVIEW TRAP One synchronous API for both editor and bulk traffic, no review routing for regulated products, and no cost estimate.

PAGE 8 OF 10

8 Behavioral Questions

role-relevant situations with STAR frameworks

STAR for this role: **S**ituation (feature, users, risk) → **T**ask (your ownership) → **A**ction (technical steps, evaluation, alternatives) → **R**esult (user metric and lasting change). Keep it to 90 seconds. Spend 60% of the time on Action.

Question	What they probe	STAR skeleton
Generated content caused a public or customer problem.	Ownership, containment, prevention.	**S:** false claim published. **T:** you owned the pipeline. **A:** paused auto-publish, traced cause, added claim check, back-scanned old outputs. **R:** zero repeats; check now a release gate.
Stakeholders judged quality by gut feel.	Turning taste into metrics.	**A:** ran a rubric workshop with copywriters, labelled 150 cases, measured agreement, built pairwise gate. **R:** shared definition; faster decisions.
Pressure to launch with a new model fast.	Risk judgment.	**A:** ran the test set in a day, found a tone regression in one language, launched for four languages only. **R:** safe launch; fix shipped a week later.
Users disliked a change that scored higher offline.	Metric alignment.	**A:** compared judge and user signals, found length bias, added length-normalised judging and acceptance gate. **R:** offline and online aligned.
Costs exceeded budget.	Trade-offs.	**A:** split cost by step, cached, batched bulk work, routed simple items to a small model, verified with pairwise eval. **R:** cost down 60%, quality flat.
Legal or brand team blocked a feature.	Collaboration.	**A:** turned their concerns into testable rules, showed violation rates, agreed review thresholds. **R:** approved launch.

What changes by level

Level	Stories they expect
Mid	You shipped a generative feature with a test set and fixed a quality issue.
Senior	You defined the rubric, the release gate, and the safety controls for a product area.
Staff	You built shared eval and prompt infrastructure used by several product teams.

Worked example: "Tell me about a quality problem you fixed"

SAMPLE ANSWER (ABOUT 80 SECONDS) **S:** Our email-draft feature had a 41% acceptance rate and users called drafts "generic". **T:** I owned the generation service. **A:** I read 200 rejected drafts and grouped the causes: no use of the thread history, wrong formality, and long openings. I built a 120-case test set and a rubric with support leads. I changed the pipeline to extract key facts from the thread first, then write with a formality field from the user's past emails, and capped length. A pairwise judge, checked against 100 human ratings (82% agreement), showed a 71% win rate. **R:** Acceptance rose to 66% in the A/B test, and the test set became the release gate.

KEY INSIGHT **What distinguishes strong:** diagnosis from real samples, a quality definition others agreed on, a measured change, and a gate that keeps the gain. **Weak:** "users liked it more".

INTERVIEW TRAP Stories where the only action is "I rewrote the prompt". Show diagnosis, a measured change, and a lasting control.

PAGE 9 OF 10

30-Minute Mock Interview

timed run with sample answers and follow-ups

Time	Segment	Interviewer is checking
0:00–2:00	Intro and motivation	Shipped generative features, not only demos.
2:00–7:00	Project walk-through	Rubric, metrics, controls, failure story.
7:00–17:00	Technical rapid-fire (Q3 to Q5)	Evaluation, structure, safety.
17:00–25:00	Design scenario (Q6)	Pipeline, two traffic paths, cost.
25:00–30:00	Your questions	Product sense and team maturity.

Q01 **Tell me about yourself and why generative AI.** 2 MIN

"I'm a full-stack engineer who has shipped two generative features: product copy and email drafts, used by about 3k people weekly. I own prompts, evals, and the safety checks. I enjoy turning subjective quality into numbers a team can act on." **Follow-up:** *What did you own personally?*

Q02 **Walk me through a generative feature you shipped.** 5 MIN

Use the seven beats from page 6. Lead with the rubric and the acceptance lift. **Follow-up:** *How did you validate the judge?*

Q03 **Acceptance dropped from 70% to 58% this week. Why?** 3 MIN

Check what changed: model version alias, prompt version, input mix (new category or language), context builder, UI. Compare outputs before and after on the test set. Read rejected samples. Roll back if a version change caused it. **Follow-up:** *Nothing changed on your side? Provider model update: pin versions and alert on output-length or tone drift.*

Q04 **How would you compare two prompts fairly?** 3 MIN

Same test set stratified by segment, several samples per input, pairwise judge with order swap, human spot-check, per-segment results, cost and latency side by side. Then an online A/B on acceptance. **Follow-up:** *Win rate is 54% on 60 cases. Ship? Not enough evidence; widen the set or check the confidence interval.*

Q05 **The generator invented a warranty term. Debug and prevent it.** 4 MIN

Find the trace: was warranty in the attributes? If not, the model filled a gap. Add a rule: warranty text only from a source field; verify in code; return "not stated". Back-scan published content. Add the case to the test set. **Follow-up:** *How do you find unknown false claims? Claim extraction plus source check on a weekly sample.*

Q06 **Design the content studio for 2M SKUs plus an editor.** 8 MIN

Follow page 7: clarify, rubric, grounding, structured pipeline, checks, two paths, capacity and cost, images, feedback. **Follow-up:** *A new language next quarter? New test set and rubric anchors per language before launch.*

Q07 **What would you do in your first 90 days?** 2 MIN

Read user feedback and rejected outputs. Build or refresh one test set and rubric. Add version tracking to every output. Ship one measured improvement.

Questions you should ask (pick two)

- How do you decide a new model or prompt is ready to ship?
- Who owns brand and safety rules, and how are they encoded?
- What user signal do you trust most for generated content?

Red flags interviewers write down

- No rubric or test set behind "better"
- Rules enforced only by prompt text
- Cannot explain judge bias
- No plan for false claims
- Cannot estimate tokens or cost
- Long answers with no structure

KEY INSIGHT **Scoring yourself:** for each answer, did you (1) define quality, (2) name a control in code, (3) cite a number, (4) stay under 90 seconds?

PAGE 10 OF 10

Final Cheat Sheet

concepts · top 15 · mistakes · checklist

Essential concepts in one glance

Control	Quality	Production
Schema first; facts from data; checks in code; bounded repair; temperature by task; pinned versions.	Rubric with anchors; pairwise gates; calibrated judges; acceptance and edit rate online.	Stream; cache; route; batch bulk work; moderation; review for high risk; cost per generation.

The 15 most important questions (answer key in one line)

#	Question	Answer in one line
1	Iterate a prompt?	Spec, test set, rubric, one change at a time, versioned.
2	Evaluate open text?	Anchored rubric, pairwise comparison, human agreement.
3	LLM-judge risks?	Position, length, self-preference; swap, calibrate, pin.
4	Reduce hallucination?	Supply facts, require references, verify in code.
5	Broken JSON?	Schema mode, simpler schema, validate, one repair.
6	On-brand and compliant?	Style guide, banned claims in code, moderation, review.
7	Image feature reliability?	Templates, seeds, automated checks, provenance, licence.
8	Feels slow?	Stream, cache prefix, cap output, parallel sections.
9	Cut cost?	Route, cache, batch, cap; confirm with pairwise eval.
10	Ship a new version?	Offline gate, safety check, canary, flag, version on output.
11	Fine-tune or prompt?	Fine-tune for style or cost with 1k+ examples; facts via retrieval.
12	Jailbreak attempts?	Moderation, rate limits, logs to tests, limits in code.
13	Temperature choice?	Low for extraction, higher for ideation; test both.
14	Offline up, online down?	Judge bias or rubric gap; align with acceptance metric.
15	Acceptance dropped?	Check version, input mix, context; pin and roll back.

Numbers worth knowing

Test set	Judge	Latency	Cost
50 to 150 cases per output type to start	Report human agreement; 80%+ is a usable bar	First token under 1 s feels responsive	Batch APIs are often about half price

Common mistakes

- "Better" with no rubric or test set
- Rules only in the prompt
- Trusting an uncalibrated judge
- Facts from model memory
- No version recorded on outputs
- Same path for bulk and interactive
- No cost per generation
- Demo screenshots as evidence

Last-minute checklist

- ☐ 90-second project story with acceptance lift
- ☐ One failure story with a code-level control
- ☐ Can sketch streaming with output checks
- ☐ Design order: clarify, rubric, grounding, checks, paths, cost
- ☐ Two questions prepared for them
- ☐ Camera, audio, this page open

KEY INSIGHT **Recruiter screen (5 to 10 min):** give a 60-second pitch (features shipped, users, one metric), confirm backend and evaluation depth, state your preferred scope (product features vs platform), and have your compensation range and notice period ready.